

```

1  import os
2  import re
3  from typing import Dict, List, Optional, Union
4  from dataclasses import dataclass, field, asdict
5  from datetime import datetime
6  import logging
7  from logging.handlers import RotatingFileHandler
8  import json
9
10 class ConfigValidationError(Exception):
11     pass
12
13 @dataclass
14 class SystemConfig:
15     software_full_name: str = "基于 YOLO 模型的显示屏表面缺陷 AI 智能检测与分析系统"
16     software_short_name: str = "显示屏表面缺陷 AI 智能系统"
17     version: str = "V1.0"
18     software_category: str = "1 应用软件"
19     development_completion_date: str = "2023 年 01 月 10 日"
20     is_published: bool = False
21     dev_hardware_env: str = "虚拟机: 16 核, 32GB 内存, 300GB 硬盘"
22     runtime_hardware_env: str = "虚拟机: 32 核, 64GB 内存, 500GB 硬盘"
23     dev_os: str = "Windows"
24     dev_environment: str = "VS2022、VSCode"
25     runtime_os: str = "Linux"
26     runtime_support_env: str = "nginx、docker、redis、PostgreSQL"
27     programming_languages: str = "Python、vue"
28     source_code_lines: int = 50000
29     development_purpose: str = "解决显示器生产企业对显示屏缺陷检测的智能化, 自动化分析问题, 提升企业生产
30 良率和出厂合格率"
31     target_industries: str = "制造业、电子产品、新型显示等行业"
32     main_functions: str = "基于 YOLO 模型实现显示屏表面缺陷的智能识别与检测, 支持缺陷类型分类、参数统计与
33 分析, 生成检测报告, 助力企业提升生产良率与出厂合格率"
34     technical_features: str = "采用 YOLO 模型实现高精度缺陷检测, 结合 Python+Vue 技术栈, 依托容器化部
35 署, 检测效率高、扩展性强"
36     company_name: str = "深圳壹视光电有限公司"
37     unified_social_credit_code: str = "91440300MA5G2XGR4U"
38     environment: str = field(default_factory=lambda: os.getenv("ENVIRONMENT",
39 "development"))
40     debug: bool = field(default_factory=lambda: os.getenv("DEBUG", "false").lower() ==
41 "true")
42     secret_key: str = field(default_factory=lambda: os.getenv("SECRET_KEY", "default-secret-
43 key-for-development-only"))
44     max_workers: int = field(default_factory=lambda: int(os.getenv("MAX_WORKERS", 8)))
45     timeout: int = field(default_factory=lambda: int(os.getenv("TIMEOUT", 30)))
46
47     def validate(self) -> None:
48         errors = []
49         if not re.match(r"^\V\d+\.\d+$", self.version):

```

```

50     errors.append("版本号格式不正确, 应为 Vx.x 格式")
51     if not re.match(r"^\d{4}年\d{2}月\d{2}日$", self.development_completion_date):
52         errors.append("开发完成日期格式不正确, 应为 YYYY 年 MM 月 DD 日")
53     if not re.match(r"^\d{17}$", self.unified_social_credit_code):
54         errors.append("统一社会信用代码格式不正确")
55     if self.source_code_lines <= 0:
56         errors.append("源程序量必须大于 0")
57     if self.max_workers <= 0:
58         errors.append("最大工作进程数必须大于 0")
59     if self.timeout <= 0:
60         errors.append("超时时间必须大于 0")
61     if not self.secret_key or len(self.secret_key) < 16:
62         errors.append("Secret Key 长度不能少于 16 个字符")
63     if errors:
64         raise ConfigValidationError("系统配置验证失败: " + "; ".join(errors))
65
66 @dataclass
67 class DatabaseConfig:
68     db_type: str = field(default_factory=lambda: os.getenv("DB_TYPE", "postgresql"))
69     host: str = field(default_factory=lambda: os.getenv("DB_HOST", "localhost"))
70     port: int = field(default_factory=lambda: int(os.getenv("DB_PORT", 5432)))
71     database: str = field(default_factory=lambda: os.getenv("DB_NAME", "display_defect_db"))
72     username: str = field(default_factory=lambda: os.getenv("DB_USER", "postgres"))
73     password: str = field(default_factory=lambda: os.getenv("DB_PASSWORD", "password"))
74     echo: bool = field(default_factory=lambda: os.getenv("DB_ECHO", "false").lower() ==
75 "true")
76     pool_size: int = field(default_factory=lambda: int(os.getenv("DB_POOL_SIZE", 10)))
77     max_overflow: int = field(default_factory=lambda: int(os.getenv("DB_MAX_OVERFLOW", 20)))
78     connect_timeout: int = field(default_factory=lambda: int(os.getenv("DB_CONNECT_TIMEOUT",
79 10)))
80     ssl_mode: str = field(default_factory=lambda: os.getenv("DB_SSL_MODE", "prefer"))
81     migrate_on_startup: bool = field(default_factory=lambda:
82 os.getenv("DB_MIGRATE_ON_STARTUP", "false").lower() == "true")
83
84     def validate(self) -> None:
85         errors = []
86         if self.db_type not in ["postgresql", "mysql", "sqlite"]:
87             errors.append("不支持的数据库类型, 仅支持 postgresql、mysql、sqlite")
88         if not self.host:
89             errors.append("数据库主机地址不能为空")
90         if not (1 <= self.port <= 65535):
91             errors.append("数据库端口必须在 1-65535 之间")
92         if not self.database:
93             errors.append("数据库名称不能为空")
94         if not self.username:
95             errors.append("数据库用户名不能为空")
96         if self.pool_size <= 0:
97             errors.append("连接池大小必须大于 0")
98         if self.max_overflow < 0:

```

```

99         errors.append("最大溢出连接数不能为负数")
100     if self.connect_timeout <= 0:
101         errors.append("数据库连接超时时间必须大于 0")
102     if self.ssl_mode not in ["disable", "allow", "prefer", "require", "verify-ca",
103 "verify-full"]:
104         errors.append("不支持的 SSL 模式")
105     if errors:
106         raise ConfigValidationError("数据库配置验证失败: " + "; ".join(errors))
107
108     def get_database_url(self) -> str:
109         if self.db_type == "sqlite":
110             return f"sqlite:///{"self.database}"
111         elif self.db_type == "mysql":
112             return f"mysql+pymysql://{self.username}:{self.password}@{self.host}:
113 {self.port}/{self.database}?charset=utf8mb4"
114         else:
115             return f"postgresql+psycpg2://{self.username}:{self.password}@{self.host}:
116 {self.port}/{self.database}?sslmode={self.ssl_mode}"
117
118 @dataclass
119 class CacheConfig:
120     cache_type: str = field(default_factory=lambda: os.getenv("CACHE_TYPE", "redis"))
121     host: str = field(default_factory=lambda: os.getenv("CACHE_HOST", "localhost"))
122     port: int = field(default_factory=lambda: int(os.getenv("CACHE_PORT", 6379)))
123     password: str = field(default_factory=lambda: os.getenv("CACHE_PASSWORD", ""))
124     database: int = field(default_factory=lambda: int(os.getenv("CACHE_DB", 0)))
125     default_timeout: int = field(default_factory=lambda:
126 int(os.getenv("CACHE_DEFAULT_TIMEOUT", 3600)))
127     max_connections: int = field(default_factory=lambda:
128 int(os.getenv("CACHE_MAX_CONNECTIONS", 10)))
129     key_prefix: str = field(default_factory=lambda: os.getenv("CACHE_KEY_PREFIX",
130 "display_defect:"))
131
132     def validate(self) -> None:
133         errors = []
134         if self.cache_type not in ["redis", "memory"]:
135             errors.append("不支持的缓存类型, 仅支持 redis 和 memory")
136         if self.cache_type == "redis":
137             if not self.host:
138                 errors.append("Redis 主机地址不能为空")
139             if not (1 <= self.port <= 65535):
140                 errors.append("Redis 端口必须在 1-65535 之间")
141             if not (0 <= self.database <= 15):
142                 errors.append("Redis 数据库编号必须在 0-15 之间")
143         if self.default_timeout <= 0:
144             errors.append("缓存默认超时时间必须大于 0")
145         if self.max_connections <= 0 and self.cache_type == "redis":
146             errors.append("Redis 最大连接数必须大于 0")
147         if errors:

```

```

148         raise ConfigValidationError("缓存配置验证失败: " + "; ".join(errors))
149
150     @dataclass
151     class LoggingConfig:
152         log_level: str = field(default_factory=lambda: os.getenv("LOG_LEVEL", "DEBUG").upper())
153         log_format: str = field(default_factory=lambda: os.getenv("LOG_FORMAT", "%(asctime)s - %(name)s - %(levelname)s - %(message)s"))
154         log_dir: str = field(default_factory=lambda: os.getenv("LOG_DIR", "logs"))
155         max_file_size: int = field(default_factory=lambda: int(os.getenv("LOG_MAX_FILE_SIZE", 104857600)))
156         backup_count: int = field(default_factory=lambda: int(os.getenv("LOG_BACKUP_COUNT", 10)))
157         enable_console_log: bool = field(default_factory=lambda: os.getenv("LOG_ENABLE_CONSOLE", "true").lower() == "true")
158         enable_file_log: bool = field(default_factory=lambda: os.getenv("LOG_ENABLE_FILE", "true").lower() == "true")
159         send_logs_to_cloud: bool = field(default_factory=lambda: os.getenv("LOG_SEND_TO_CLOUD", "false").lower() == "true")
160
161     def validate(self) -> None:
162         errors = []
163         valid_levels = ["DEBUG", "INFO", "WARNING", "ERROR", "CRITICAL"]
164         if self.log_level not in valid_levels:
165             errors.append(f"日志级别无效, 必须是{'', '.join(valid_levels)}之一")
166         if self.max_file_size <= 0:
167             errors.append("日志文件最大容量必须大于 0")
168         if self.backup_count < 0:
169             errors.append("日志备份数量不能为负数")
170         if not self.enable_console_log and not self.enable_file_log and not self.send_logs_to_cloud:
171             errors.append("至少要启用一种日志输出方式")
172         if errors:
173             raise ConfigValidationError("日志配置验证失败: " + "; ".join(errors))
174
175     def setup_logging(self) -> None:
176         os.makedirs(self.log_dir, exist_ok=True)
177         logger = logging.getLogger()
178         logger.setLevel(getattr(logging, self.log_level))
179
180         formatter = logging.Formatter(self.log_format)
181
182         if self.enable_console_log:
183             console_handler = logging.StreamHandler()
184             console_handler.setFormatter(formatter)
185             logger.addHandler(console_handler)
186
187         if self.enable_file_log:
188             file_handler = RotatingFileHandler(
189                 os.path.join(self.log_dir, "app.log"),

```

```

197         maxBytes=self.max_file_size,
198         backupCount=self.backup_count,
199         encoding="utf-8"
200     )
201     file_handler.setFormatter(formatter)
202     logger.addHandler(file_handler)
203
204     if self.send_logs_to_cloud:
205         pass
206
207 @dataclass
208 class APIConfig:
209     api_host: str = field(default_factory=lambda: os.getenv("API_HOST", "0.0.0.0"))
210     api_port: int = field(default_factory=lambda: int(os.getenv("API_PORT", 8000)))
211     api_prefix: str = field(default_factory=lambda: os.getenv("API_PREFIX", "/api/v1"))
212     enable_cors: bool = field(default_factory=lambda: os.getenv("ENABLE_CORS",
213 "true").lower() == "true")
214     allowed_origins: List[str] = field(default_factory=lambda:
215 json.loads(os.getenv("ALLOWED_ORIGINS", '["*"]')))
216     rate_limit: str = field(default_factory=lambda: os.getenv("RATE_LIMIT", "100/minute"))
217     enable_swagger: bool = field(default_factory=lambda: os.getenv("ENABLE_SWAGGER",
218 "true").lower() == "true")
219     enable_metrics: bool = field(default_factory=lambda: os.getenv("ENABLE_METRICS",
220 "true").lower() == "true")
221     metrics_port: int = field(default_factory=lambda: int(os.getenv("METRICS_PORT", 9090)))
222
223     def validate(self) -> None:
224         errors = []
225         if not re.match(r"^\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}$|^0\.0\.0\.0$|^localhost$",
226 self.api_host):
227             errors.append("API 主机地址格式不正确")
228         if not (1 <= self.api_port <= 65535):
229             errors.append("API 端口必须在 1-65535 之间")
230         if not self.api_prefix.startswith("/"):
231             errors.append("API 前缀必须以 / 开头")
232         if self.enable_cors and not self.allowed_origins:
233             errors.append("启用 CORS 时必须配置允许的源")
234         if not re.match(r"^\d+/(minute|hour|day)$", self.rate_limit):
235             errors.append("速率限制格式不正确，应为数字/(minute|hour|day)")
236         if self.enable_metrics and not (1 <= self.metrics_port <= 65535):
237             errors.append("Metrics 端口必须在 1-65535 之间")
238         if errors:
239             raise ConfigValidationError("API 配置验证失败: " + "; ".join(errors))
240
241 @dataclass
242 class YOLOConfig:
243     model_path: str = field(default_factory=lambda: os.getenv("YOLO_MODEL_PATH",
244 "models/yolov8n-defect.pt"))

```

```

245     confidence_threshold: float = field(default_factory=lambda:
246 float(os.getenv("YOLO_CONFIDENCE", 0.7)))
247     iou_threshold: float = field(default_factory=lambda: float(os.getenv("YOLO_IOU", 0.5)))
248     defect_types: List[str] = field(default_factory=lambda:
249 json.loads(os.getenv("DEFECT_TYPES", '["划痕", "气泡", "漏光", "脏污", "破裂"]'))
250     max_det: int = field(default_factory=lambda: int(os.getenv("YOLO_MAX_DET", 100)))
251     device: str = field(default_factory=lambda: os.getenv("YOLO_DEVICE", "cpu"))
252     enable_batch_processing: bool = field(default_factory=lambda:
253 os.getenv("YOLO_BATCH_PROCESS", "true").lower() == "true")
254     batch_size: int = field(default_factory=lambda: int(os.getenv("YOLO_BATCH_SIZE", 8)))
255
256     def validate(self) -> None:
257         errors = []
258         if not os.path.exists(self.model_path) and not self.model_path.startswith("yolov"):
259             errors.append(f"YOLO 模型文件不存在: {self.model_path}")
260         if not (0 < self.confidence_threshold <= 1):
261             errors.append("置信度阈值必须在 0-1 之间")
262         if not (0 < self.iou_threshold <= 1):
263             errors.append("IOU 阈值必须在 0-1 之间")
264         if not self.defect_types:
265             errors.append("缺陷类型列表不能为空")
266         if self.max_det <= 0:
267             errors.append("最大检测数量必须大于 0")
268         if self.device not in ["cpu", "cuda", "mps"]:
269             errors.append("不支持的设备类型, 仅支持 cpu、cuda、mps")
270         if self.enable_batch_processing and self.batch_size <= 0:
271             errors.append("批处理大小必须大于 0")
272         if errors:
273             raise ConfigValidationError("YOLO 配置验证失败: " + "; ".join(errors))
274
275 @dataclass
276 class AppConfig:
277     system: SystemConfig = field(default_factory=SystemConfig)
278     database: DatabaseConfig = field(default_factory=DatabaseConfig)
279     cache: CacheConfig = field(default_factory=CacheConfig)
280     logging: LoggingConfig = field(default_factory=LoggingConfig)
281     api: APIConfig = field(default_factory=APIConfig)
282     yolo: YOLOConfig = field(default_factory=YOLOConfig)
283
284     def load_from_env(self) -> None:
285         self.system = SystemConfig()
286         self.database = DatabaseConfig()
287         self.cache = CacheConfig()
288         self.logging = LoggingConfig()
289         self.api = APIConfig()
290         self.yolo = YOLOConfig()
291
292     def load_from_file(self, config_file: str) -> None:
293         if not os.path.exists(config_file):

```

```

294         raise FileNotFoundError(f"配置文件不存在: {config_file}")
295
296     with open(config_file, "r", encoding="utf-8") as f:
297         config_data = json.load(f)
298
299     if "system" in config_data:
300         self.system = SystemConfig(**config_data["system"])
301     if "database" in config_data:
302         self.database = DatabaseConfig(**config_data["database"])
303     if "cache" in config_data:
304         self.cache = CacheConfig(**config_data["cache"])
305     if "logging" in config_data:
306         self.logging = LoggingConfig(**config_data["logging"])
307     if "api" in config_data:
308         self.api = APIConfig(**config_data["api"])
309
310
311 from datetime import datetime
312 from typing import List, Optional, Dict, Any
313 from pydantic import BaseModel, Field, validator, ValidationError
314 import psycpg2
315 from psycpg2.extras import RealDictCursor
316 from enum import Enum
317 import json
318
319 class UserRoleEnum(str, Enum):
320     ADMIN = "admin"
321     OPERATOR = "operator"
322     VIEWER = "viewer"
323
324 class DefectTypeEnum(str, Enum):
325     SCRATCH = "scratch"
326     CRACK = "crack"
327     DIRT = "dirt"
328     DISCOLORATION = "discoloration"
329     MURA = "mura"
330     OTHER = "other"
331
332 class User(BaseModel):
333     id: Optional[int] = None
334     username: str = Field(..., min_length=3, max_length=50)
335     password_hash: str = Field(..., min_length=60)
336     real_name: str = Field(..., min_length=2, max_length=50)
337     role: UserRoleEnum = Field(...)
338     email: Optional[str] = Field(None, max_length=100)
339     phone: Optional[str] = Field(None, max_length=20)
340     company_id: int = Field(...)
341     created_at: Optional[datetime] = None
342     updated_at: Optional[datetime] = None

```

```

343
344 @validator('email')
345 def validate_email(cls, v):
346     if v and '@' not in v:
347         raise ValueError('Invalid email format')
348     return v
349
350 @validator('phone')
351 def validate_phone(cls, v):
352     if v and not v.isdigit():
353         raise ValueError('Phone number must contain only digits')
354     return v
355
356 def to_db_dict(self) -> Dict[str, Any]:
357     data = self.dict(exclude_unset=True)
358     if self.created_at:
359         data['created_at'] = self.created_at.isoformat()
360     if self.updated_at:
361         data['updated_at'] = self.updated_at.isoformat()
362     return data
363
364 @classmethod
365 def from_db_dict(cls, db_data: Dict[str, Any]) -> 'User':
366     db_data['created_at'] = datetime.fromisoformat(db_data['created_at'])
367     db_data['updated_at'] = datetime.fromisoformat(db_data['updated_at'])
368     return cls(**db_data)
369
370 @classmethod
371 def get_by_id(cls, db_conn, user_id: int) -> Optional['User']:
372     cursor = db_conn.cursor(cursor_factory=RealDictCursor)
373     try:
374         cursor.execute("SELECT * FROM users WHERE id = %s", (user_id,))
375         result = cursor.fetchone()
376         if result:
377             return cls.from_db_dict(result)
378         return None
379     finally:
380         cursor.close()
381
382 @classmethod
383 def get_by_username(cls, db_conn, username: str) -> Optional['User']:
384     cursor = db_conn.cursor(cursor_factory=RealDictCursor)
385     try:
386         cursor.execute("SELECT * FROM users WHERE username = %s", (username,))
387         result = cursor.fetchone()
388         if result:
389             return cls.from_db_dict(result)
390         return None
391     finally:

```

```

392         cursor.close()
393
394     def save(self, db_conn) -> int:
395         cursor = db_conn.cursor()
396         try:
397             if self.id:
398                 cursor.execute("""
399                     UPDATE users SET username = %s, password_hash = %s, real_name = %s, role
400 = %s,
401                     email = %s, phone = %s, company_id = %s, updated_at = %s
402                     WHERE id = %s RETURNING id
403                     """, (self.username, self.password_hash, self.real_name, self.role.value,
404                         self.email, self.phone, self.company_id, datetime.now(), self.id))
405             else:
406                 cursor.execute("""
407                     INSERT INTO users (username, password_hash, real_name, role, email,
408 phone,
409                     company_id, created_at, updated_at)
410                     VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s) RETURNING id
411                     """, (self.username, self.password_hash, self.real_name, self.role.value,
412                         self.email, self.phone, self.company_id, datetime.now(),
413 datetime.now()))
414                 self.id = cursor.fetchone()[0]
415                 db_conn.commit()
416                 return self.id
417             except Exception as e:
418                 db_conn.rollback()
419                 raise e
420             finally:
421                 cursor.close()
422
423     class Company(BaseModel):
424         id: Optional[int] = None
425         name: str = Field(..., min_length=2, max_length=100)
426         unified_social_credit_code: str = Field(..., min_length=18, max_length=18)
427         address: Optional[str] = Field(None, max_length=200)
428         contact_phone: Optional[str] = Field(None, max_length=20)
429         created_at: Optional[datetime] = None
430         updated_at: Optional[datetime] = None
431
432         @validator('unified_social_credit_code')
433         def validate_credit_code(cls, v):
434             if not v.isalnum() or len(v) != 18:
435                 raise ValueError('Invalid unified social credit code')
436             return v
437
438         def to_db_dict(self) -> Dict[str, Any]:
439             data = self.dict(exclude_unset=True)
440             if self.created_at:

```

```

441         data['created_at'] = self.created_at.isoformat()
442     if self.updated_at:
443         data['updated_at'] = self.updated_at.isoformat()
444     return data
445
446     @classmethod
447     def from_db_dict(cls, db_data: Dict[str, Any]) -> 'Company':
448         db_data['created_at'] = datetime.fromisoformat(db_data['created_at'])
449         db_data['updated_at'] = datetime.fromisoformat(db_data['updated_at'])
450         return cls(**db_data)
451
452     @classmethod
453     def get_by_id(cls, db_conn, company_id: int) -> Optional['Company']:
454         cursor = db_conn.cursor(cursor_factory=RealDictCursor)
455         try:
456             cursor.execute("SELECT * FROM companies WHERE id = %s", (company_id,))
457             result = cursor.fetchone()
458             if result:
459                 return cls.from_db_dict(result)
460             return None
461         finally:
462             cursor.close()
463
464     @classmethod
465     def get_by_credit_code(cls, db_conn, credit_code: str) -> Optional['Company']:
466         cursor = db_conn.cursor(cursor_factory=RealDictCursor)
467         try:
468             cursor.execute("SELECT * FROM companies WHERE unified_social_credit_code = %s",
469 (credit_code,))
470             result = cursor.fetchone()
471             if result:
472                 return cls.from_db_dict(result)
473             return None
474         finally:
475             cursor.close()
476
477     def save(self, db_conn) -> int:
478         cursor = db_conn.cursor()
479         try:
480             if self.id:
481                 cursor.execute("""
482                     UPDATE companies SET name = %s, unified_social_credit_code = %s, address
483 = %s,
484                     contact_phone = %s, updated_at = %s WHERE id = %s RETURNING id
485                     """, (self.name, self.unified_social_credit_code, self.address,
486                         self.contact_phone, datetime.now(), self.id))
487             else:
488                 cursor.execute("""

```

```

489         INSERT INTO companies (name, unified_social_credit_code, address,
490 contact_phone,
491         created_at, updated_at)
492         VALUES (%s, %s, %s, %s, %s, %s) RETURNING id
493         """', (self.name, self.unified_social_credit_code, self.address,
494         self.contact_phone, datetime.now(), datetime.now()))
495         self.id = cursor.fetchone()[0]
496         db_conn.commit()
497         return self.id
498     except Exception as e:
499         db_conn.rollback()
500         raise e
501     finally:
502         cursor.close()
503
504 class Project(BaseModel):
505     id: Optional[int] = None
506     name: str = Field(..., min_length=2, max_length=100)
507     description: Optional[str] = Field(None, max_length=500)
508     company_id: int = Field(...)
509     yolo_model_version: str = Field(..., min_length=1, max_length=20)
510     created_by: int = Field(...)
511     created_at: Optional[datetime] = None
512     updated_at: Optional[datetime] = None
513
514     def to_db_dict(self) -> Dict[str, Any]:
515         data = self.dict(exclude_unset=True)
516         if self.created_at:
517             data['created_at'] = self.created_at.isoformat()
518         if self.updated_at:
519             data['updated_at'] = self.updated_at.isoformat()
520         return data
521
522     @classmethod
523     def from_db_dict(cls, db_data: Dict[str, Any]) -> 'Project':
524         db_data['created_at'] = datetime.fromisoformat(db_data['created_at'])
525         db_data['updated_at'] = datetime.fromisoformat(db_data['updated_at'])
526         return cls(**db_data)
527
528     @classmethod
529     def get_by_id(cls, db_conn, project_id: int) -> Optional['Project']:
530         cursor = db_conn.cursor(cursor_factory=RealDictCursor)
531         try:
532             cursor.execute("SELECT * FROM projects WHERE id = %s", (project_id,))
533             result = cursor.fetchone()
534             if result:
535                 return cls.from_db_dict(result)
536             return None
537         finally:

```

```

538         cursor.close()
539
540     @classmethod
541     def get_by_company(cls, db_conn, company_id: int) -> List['Project']:
542         cursor = db_conn.cursor(cursor_factory=RealDictCursor)
543         try:
544             cursor.execute("SELECT * FROM projects WHERE company_id = %s ORDER BY created_at
545 DESC", (company_id,))
546             results = cursor.fetchall()
547             return [cls.from_db_dict(res) for res in results]
548         finally:
549             cursor.close()
550
551     def save(self, db_conn) -> int:
552         cursor = db_conn.cursor()
553         try:
554             if self.id:
555                 cursor.execute("""
556                     UPDATE projects SET name = %s, description = %s, company_id = %s,
557                         yolo_model_version = %s, updated_at = %s WHERE id = %s RETURNING id
558                     """, (self.name, self.description, self.company_id, self.yolo_model_version,
559                         datetime.now(), self.id))
560             else:
561                 cursor.execute("""
562                     INSERT INTO projects (name, description, company_id, yolo_model_version,
563                         created_by, created_at, updated_at)
564                     VALUES (%s, %s, %s, %s, %s, %s, %s) RETURNING id
565                     """, (self.name, self.description, self.company_id, self.yolo_model_version,
566                         self.created_by, datetime.now(), datetime.now()))
567             self.id = cursor.fetchone()[0]
568             db_conn.commit()
569             return self.id
570         except Exception as e:
571             db_conn.rollback()
572             raise e
573         finally:
574             cursor.close()
575
576 class DefectDetectionRecord(BaseModel):
577     id: Optional[int] = None
578     project_id: int = Field(...)
579     screen_serial_number: str = Field(..., min_length=5, max_length=50)
580     detection_time: datetime = Field(...)
581     defect_types: List[DefectTypeEnum] = Field(...)
582     defect_count: int = Field(..., ge=0)
583     defect_details: List[Dict[str, Any]] = Field(...)
584     confidence_score: float = Field(..., ge=0.0, le=1.0)
585     is_qualified: bool = Field(...)
586     detected_by: int = Field(...)

```

```

587 image_path: str = Field(..., min_length=10, max_length=200)
588 report_path: Optional[str] = Field(None, max_length=200)
589 created_at: Optional[datetime] = None
590 updated_at: Optional[datetime] = None
591
592 @validator('defect_details')
593 def validate_defect_details(cls, v):
594     for detail in v:
595         if 'type' not in detail or 'confidence' not in detail or 'bbox' not in detail:
596             raise ValueError('Invalid defect detail format')
597         if not isinstance(detail['confidence'], float) or detail['confidence'] < 0 or
598 detail['confidence'] > 1:
599             raise ValueError('Defect confidence must be between 0 and 1')
600         if not isinstance(detail['bbox'], list) or len(detail['bbox']) != 4:
601             raise ValueError('Defect bbox must be a list of 4 numbers')
602     return v
603
604 def to_db_dict(self) -> Dict[str, Any]:
605     data = self.dict(exclude_unset=True)
606     data['defect_types'] = [dt.value for dt in self.defect_types]
607     data['defect_details'] = json.dumps(self.defect_details)
608     data['detection_time'] = self.detection_time.isoformat()
609     if self.created_at:
610         data['created_at'] = self.created_at.isoformat()
611     if self.updated_at:
612         data['updated_at'] = self.updated_at.isoformat()
613     return data
614
615 @classmethod
616 def from_db_dict(cls, db_data: Dict[str, Any]) -> 'DefectDetectionRecord':
617     db_data['defect_types'] = [DefectTypeEnum(dt) for dt in db_data['defect_types']]
618     db_data['defect_details'] = json.loads(db_data['defect_details'])
619     db_data['detection_time'] = datetime.fromisoformat(db_data['detection_time'])
620     db_data['created_at'] = datetime.fromisoformat(db_data['created_at'])
621     db_data['updated_at'] = datetime.fromisoformat(db_data['updated_at'])
622     return cls(**db_data)
623
624 @classmethod
625 def get_by_id(cls, db_conn, record_id: int) -> Optional['DefectDetectionRecord']:
626     cursor = db_conn.cursor(cursor_factory=RealDictCursor)
627     try:
628         cursor.execute("SELECT * FROM defect_detection_records WHERE id = %s",
629 (record_id,))
630         result = cursor.fetchone()
631         if result:
632             return cls.from_db_dict(result)
633         return None
634     finally:
635         cursor.close()

```

```

636
637     @classmethod
638     def get_by_project(cls, db_conn, project_id: int, start_time: Optional[datetime] = None,
639                       end_time: Optional[datetime] = None, is_qualified: Optional[bool] =
640 None) -> List['DefectDetectionRecord']:
641         cursor = db_conn.cursor(cursor_factory=RealDictCursor)
642         try:
643             query = "SELECT * FROM defect_detection_records WHERE project_id = %s"
644             params = [project_id]
645             if start_time:
646                 query += " AND detection_time >= %s"
647                 params.append(start_time.isoformat())
648             if end_time:
649                 query += " AND detection_time <= %s"
650                 params.append(end_time.isoformat())
651             if is_qualified is not None:
652                 query += " AND is_qualified = %s"
653                 params.append(is_qualified)
654             query += " ORDER BY detection_time DESC"
655             cursor.execute(query, tuple(params))
656             results = cursor.fetchall()
657             return [cls.from_db_dict(res) for res in results]
658         finally:
659             cursor.close()
660
661     @classmethod
662     def get_statistics_by_project(cls, db_conn, project_id: int, start_time:
663 Optional[datetime] = None,
664                                end_time: Optional[datetime] = None) -> Dict[str, Any]:
665         cursor = db_conn.cursor(cursor_factory=RealDictCursor)
666         try:
667             query = """
668             SELECT
669                 COUNT(*) as total_records,
670                 SUM(CASE WHEN is_qualified THEN 1 ELSE 0 END) as qualified_count,
671                 SUM(CASE WHEN NOT is_qualified THEN 1 ELSE 0 END) as unqualified_count,
672                 AVG(defect_count) as avg_defect_count,
673                 json_agg(DISTINCT defect_types) as all_defect_types
674             FROM defect_detection_records
675             WHERE project_id = %s
676             """
677             params = [project_id]
678             if start_time:
679                 query += " AND detection_time >= %s"
680                 params.append(start_time.isoformat())
681             if end_time:
682                 query += " AND detection_time <= %s"
683                 params.append(end_time.isoformat())
684             cursor.execute(query, tuple(params))

```

```

685         result = cursor.fetchone()
686         if not result:
687             return {
688                 "total_records": 0,
689                 "qualified_count": 0,
690                 "unqualified_count": 0,
691                 "avg_defect_count": 0.0,
692                 "all_defect_types": []
693             }
694         return {
695             "total_records": result['total_records'],
696             "qualified_count": result['qualified_count'] or 0,
697             "unqualified_count": result['unqualified_count'] or 0,
698             "avg_defect_count": result['avg_defect_count'] or 0.0,
699             "all_defect_types": result['all_defect_types'] or []
700         }
701     finally:
702         cursor.close()
703
704     def save(self, db_conn) -> int:
705         cursor = db_conn.cursor()
706         try:
707             if self.id:
708                 cursor.execute("""
709                     UPDATE defect_detection_records SET project_id = %s,
710 screen_serial_number = %s,
711                     detection_time = %s, defect_types = %s, defect_count = %s,
712                     defect_details = %s, confidence_score = %s, is_qualified = %s,
713                     detected_by = %s, image_path = %s
714
715 import os
716 import re
717 import json
718 import time
719 import hashlib
720 import base64
721 import logging
722 import traceback
723 from datetime import datetime, timedelta
724 from typing import Any, Dict, List, Optional, Union
725 from functools import wraps
726
727 def str_to_bool(s: Union[str, bool]) -> bool:
728     if isinstance(s, bool):
729         return s
730     s = s.strip().lower()
731     return s in ('true', '1', 'yes', 'y', '是')
732
733 def str_to_int(s: str, default: int = 0) -> int:

```

```

734     try:
735         return int(s.strip())
736     except (ValueError, AttributeError):
737         return default
738
739 def str_to_float(s: str, default: float = 0.0) -> float:
740     try:
741         return float(s.strip())
742     except (ValueError, AttributeError):
743         return default
744
745 def clean_string(s: str) -> str:
746     if not isinstance(s, str):
747         return ""
748     s = re.sub(r'\s+', ' ', s.strip())
749     return s
750
751 def truncate_string(s: str, max_length: int = 100, suffix: str = '...') -> str:
752     s = clean_string(s)
753     if len(s) <= max_length:
754         return s
755     return s[:max_length - len(suffix)] + suffix
756
757 def camel_to_snake(name: str) -> str:
758     name = re.sub('([A-Z][a-z]+)', r'\1_\2', name)
759     return re.sub('([a-z0-9])([A-Z])', r'\1_\2', name).lower()
760
761 def snake_to_camel(name: str, upper_first: bool = False) -> str:
762     components = name.split('_')
763     if upper_first:
764         return ''.join(x.title() for x in components)
765     return components[0] + ''.join(x.title() for x in components[1:])
766
767 def format_datetime(dt: Union[datetime, str], fmt: str = "%Y-%m-%d %H:%M:%S") -> str:
768     if isinstance(dt, str):
769         try:
770             dt = datetime.fromisoformat(dt.replace('Z', '+00:00'))
771         except ValueError:
772             try:
773                 dt = datetime.strptime(dt, "%Y-%m-%d %H:%M:%S")
774             except ValueError:
775                 return ""
776     return dt.strftime(fmt) if isinstance(dt, datetime) else ""
777
778 def parse_datetime(s: str) -> Optional[datetime]:
779     formats = [
780         "%Y-%m-%d %H:%M:%S",
781         "%Y-%m-%d",
782         "%Y/%m/%d %H:%M:%S",

```

```

783         "%Y/%m/%d",
784         "%Y-%m-%dT%H:%M:%SZ",
785         "%Y-%m-%dT%H:%M:%S.%fZ"
786     ]
787     for fmt in formats:
788         try:
789             return datetime.strptime(s, fmt)
790         except ValueError:
791             continue
792     try:
793         return datetime.fromisoformat(s.replace('Z', '+00:00'))
794     except ValueError:
795         return None
796
797 def get_current_datetime(fmt: Optional[str] = None) -> Union[str, datetime]:
798     now = datetime.now()
799     return now.strftime(fmt) if fmt else now
800
801 def get_date_diff(d1: Union[datetime, str], d2: Union[datetime, str], unit: str = "days") ->
802 float:
803     dt1 = parse_datetime(d1) if isinstance(d1, str) else d1
804     dt2 = parse_datetime(d2) if isinstance(d2, str) else d2
805     if not dt1 or not dt2:
806         return 0.0
807     diff = abs(dt1 - dt2)
808     if unit == "seconds":
809         return diff.total_seconds()
810     elif unit == "minutes":
811         return diff.total_seconds() / 60
812     elif unit == "hours":
813         return diff.total_seconds() / 3600
814     elif unit == "days":
815         return diff.days
816     elif unit == "weeks":
817         return diff.days / 7
818     elif unit == "months":
819         return (dt1.year - dt2.year) * 12 + (dt1.month - dt2.month)
820     elif unit == "years":
821         return dt1.year - dt2.year + (dt1.month - dt2.month)/12
822     return 0.0
823
824 def add_days_to_date(date: Union[datetime, str], days: int) -> str:
825     dt = parse_datetime(date) if isinstance(date, str) else date
826     if not dt:
827         return ""
828     new_dt = dt + timedelta(days=days)
829     return new_dt.strftime("%Y-%m-%d")
830
831 def ensure_dir_exists(dir_path: str) -> bool:

```

```
832     try:
833         os.makedirs(dir_path, exist_ok=True)
834         return True
835     except OSError:
836         return False
837
838 def get_file_extension(file_path: str) -> str:
839     return os.path.splitext(file_path)[1].lower()[1:]
840
841 def get_file_size(file_path: str, unit: str = "bytes") -> float:
842     if not os.path.exists(file_path) or not os.path.isfile(file_path):
843         return 0.0
844     size = os.path.getsize(file_path)
845     if unit == "kb":
846         return size / 1024
847     elif unit == "mb":
848         return size / (1024 * 1024)
849     elif unit == "gb":
850         return size / (1024 * 1024 * 1024)
851     return float(size)
852
853 def read_file_content(file_path: str, encoding: str = "utf-8") -> str:
854     try:
855         with open(file_path, 'r', encoding=encoding) as f:
856             return f.read()
857     except Exception:
858         return ""
859
860 def write_file_content(file_path: str, content: str, encoding: str = "utf-8", append: bool =
861 False) -> bool:
862     mode = 'a' if append else 'w'
863     try:
864         ensure_dir_exists(os.path.dirname(file_path))
865         with open(file_path, mode, encoding=encoding) as f:
866             f.write(content)
867         return True
868     except Exception:
869         return False
870
871 def read_json_file(file_path: str) -> Dict[str, Any]:
872     try:
873         with open(file_path, 'r', encoding='utf-8') as f:
874             return json.load(f)
875     except Exception:
876         return {}
877
878 def write_json_file(file_path: str, data: Dict[str, Any], indent: int = 2) -> bool:
879     try:
880         ensure_dir_exists(os.path.dirname(file_path))
```

```

881         with open(file_path, 'w', encoding='utf-8') as f:
882             json.dump(data, f, ensure_ascii=False, indent=indent)
883         return True
884     except Exception:
885         return False
886
887 def hash_string(s: str, algorithm: str = "md5") -> str:
888     s = s.encode('utf-8')
889     if algorithm == "md5":
890         return hashlib.md5(s).hexdigest()
891     elif algorithm == "sha1":
892         return hashlib.sha1(s).hexdigest()
893     elif algorithm == "sha256":
894         return hashlib.sha256(s).hexdigest()
895     elif algorithm == "sha512":
896         return hashlib.sha512(s).hexdigest()
897     return hashlib.md5(s).hexdigest()
898
899 def encrypt_aes(s: str, key: str) -> str:
900     from cryptography.fernet import Fernet
901     key_hash = hashlib.sha256(key.encode()).digest()
902     fernet = Fernet(base64.urlsafe_b64encode(key_hash))
903     return fernet.encrypt(s.encode()).decode()
904
905 def decrypt_aes(encrypted_str: str, key: str) -> str:
906     from cryptography.fernet import Fernet
907     key_hash = hashlib.sha256(key.encode()).digest()
908     fernet = Fernet(base64.urlsafe_b64encode(key_hash))
909     try:
910         return fernet.decrypt(encrypted_str.encode()).decode()
911     except Exception:
912         return ""
913
914 def validate_email(email: str) -> bool:
915     pattern = r'^[a-zA-Z0-9_+@][a-zA-Z0-9-]+\.[a-zA-Z0-9-]+$'
916     return re.match(pattern, email) is not None
917
918 def validate_phone(phone: str) -> bool:
919     pattern = r'^1[3-9]\d{9}$'
920     return re.match(pattern, phone.strip()) is not None
921
922 def validate_id_card(id_card: str) -> bool:
923     id_card = clean_string(id_card)
924     if len(id_card) != 18:
925         return False
926     if not re.match(r'^\d{17}[\dXx]$', id_card):
927         return False
928     coefficients = [7, 9, 10, 5, 8, 4, 2, 1, 6, 3, 7, 9, 10, 5, 8, 4, 2]
929     check_codes = ['1', '0', 'X', '9', '8', '7', '6', '5', '4', '3', '2']

```

```

930     total = sum([coefficients[i] * int(id_card[i]) for i in range(17)])
931     return check_codes[total % 11] == id_card[-1].upper()
932
933 def validate_credit_code(code: str) -> bool:
934     code = clean_string(code)
935     if len(code) != 18:
936         return False
937     if not re.match(r'^[0-9A-Z]{18}$', code):
938         return False
939     base_code = '0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ'
940     weights = [1,3,9,27,19,26,16,17,20,29,25,13,8,24,10,30,28]
941     total = 0
942     for i in range(17):
943         try:
944             index = base_code.index(code[i])
945             total += index * weights[i]
946         except ValueError:
947             return False
948     remainder = total % 31
949     check_code = base_code[31 - remainder] if remainder != 0 else base_code[0]
950     return code[-1] == check_code
951
952 def format_number(num: Union[int, float], decimal_places: int = 2) -> str:
953     try:
954         num = float(num)
955         return f"{num:.{decimal_places}f}"
956     except (ValueError, TypeError):
957         return "0.00"
958
959 def format_currency(amount: Union[int, float], currency: str = "¥") -> str:
960     amount_str = format_number(amount, 2)
961     return f"{currency}{amount_str}"
962
963 def format_percentage(num: Union[int, float], decimal_places: int = 2) -> str:
964     try:
965         num = float(num) * 100
966         return f"{num:.{decimal_places}f}%"
967     except (ValueError, TypeError):
968         return "0.00%"
969
970 def dict_to_query_string(params: Dict[str, Any]) -> str:
971     query_parts = []
972     for key, value in params.items():
973         if value is None:
974             continue
975         if isinstance(value, bool):
976             value = str(value).lower()
977         elif isinstance(value, (int, float)):
978             value = str(value)

```

```

979         query_parts.append(f"{key}={value}")
980     return '&'.join(query_parts)
981
982 def query_string_to_dict(query_str: str) -> Dict[str, str]:
983     params = {}
984     if not query_str:
985         return params
986     if query_str.startswith('?'):
987         query_str = query_str[1:]
988     for part in query_str.split('&'):
989         if '=' not in part:
990             continue
991         key, value = part.split('=', 1)
992         params[key] = value
993     return params
994
995 def deep_merge_dict(a: Dict[str, Any], b: Dict[str, Any]) -> Dict[str, Any]:
996     merged = a.copy()
997     for key, value in b.items():
998         if key in merged and isinstance(merged[key], dict) and isinstance(value, dict):
999             merged[key] = deep_merge_dict(merged[key], value)
1000         else:
1001             merged[key] = value
1002     return merged
1003
1004 def get_nested_value(data: Dict[str, Any], path: str, default: Any = None) -> Any:
1005     keys = path.split('.')
1006     current = data
1007     try:
1008         for key in keys:
1009             current = current[key]
1010         return current
1011     except (KeyError, TypeError, IndexError):
1012         return default
1013
1014 def setup_logger(name: str = "default", log_file: Optional[str] = None, level: int =
1015 logging.INFO) -> logging.Logger:
1016     logger = logging.getLogger(name)
1017     logger.setLevel(level)
1018     if logger.handlers:
1019         return logger
1020
1021     formatter = logging.Formatter('%(asctime)s - %(name)s - %(levelname)s - %(message)s')
1022
1023     console_handler = logging.StreamHandler()
1024     console_handler.setFormatter(formatter)
1025     logger.addHandler(console_handler)
1026
1027     if log_file:

```

```

1028     ensure_dir_exists(os.path.dirname(log_file))
1029     file_handler = logging.FileHandler(log_file, encoding='utf-8')
1030     file_handler.setFormatter(formatter)
1031     logger.addHandler(file_handler)
1032
1033     return logger
1034
1035 def catch_exceptions(default_return: Any = None, logger: Optional[logging.Logger] = None):
1036     def decorator(func):
1037         @wraps(func)
1038         def wrapper(*args, **kwargs):
1039             try:
1040                 return func(*args, **kwargs)
1041             except Exception as e:
1042                 error_msg = f"Exception in {func.__name__}: {str(e)}"
1043                 \n{traceback.format_exc()}
1044                 if logger:
1045                     logger.error(error_msg)
1046                 else:
1047                     print(error_msg)
1048                 return default_return
1049             return wrapper
1050     return decorator
1051
1052 def retry(max_attempts: int = 3, delay: float = 1.0, exceptions: tuple = (Exception,),
1053 logger: Optional[logging.Logger] = None):
1054     def decorator(func):
1055         @wraps(func)
1056         def wrapper(*args, **kwargs):
1057             attempts = 0
1058             while attempts < max_attempts:
1059                 try:
1060                     return func(*args, **kwargs)
1061                 except exceptions as e:
1062                     attempts += 1
1063                     error_msg = f"Attempt {attempts} failed in {func.__name__}: {str(e)}"
1064                     if logger:
1065                         logger.warning(error_msg)
1066                     else:
1067                         print(error_msg)
1068                     if attempts < max_attempts:
1069                         time.sleep(delay)
1070             error_msg = f"All {max_attempts} attempts failed in {func.__name__}"
1071             if logger:
1072                 logger.error(error_msg)
1073             else:
1074                 print(error_msg)
1075             raise Exception(error_msg)
1076     return wrapper

```

```

1077     return decorator
1078
1079 def generate_random_string(length: int = 16, include_numbers: bool = True,
1080 include_uppercase: bool = True, include_special: bool = False) -> str:
1081     import random
1082     import string
1083     chars = string.ascii_lowercase
1084     if include_uppercase:
1085         chars += string.ascii_uppercase
1086     if include_numbers:
1087         chars += string.digits
1088     if include_special:
1089         chars += string.punctuation
1090     return ''.join(random.choice(chars) for _ in range(length))
1091
1092 def get_ip_address() -> str:
1093     import socket
1094     try:
1095         s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
1096         s.connect(("8.8.8.8", 80))
1097         ip = s.getsockname()[0]
1098         s.close()
1099         return ip
1100     except Exception:
1101         return "127.0.0.1"
1102
1103 def get_system_info() -> Dict[str, Any]:
1104     import platform
1105     return {
1106         "os": platform.system(),
1107         "os_version": platform.version(),
1108         "platform": platform.platform(),
1109         "python_version": platform.python_version(),
1110         "machine": platform.machine(),
1111         "processor": platform.processor()
1112     }
1113
1114 def convert_bytes_to_human_readable(size: int) -> str:
1115     for unit in ['B', 'KB', 'MB', 'GB', 'TB']:
1116         if size < 1024.0:
1117             return f"{size:.2f} {unit}"
1118         size /= 1024.0
1119     return f"{size:.2f} PB"
1120
1121 def remove_duplicates_from_list(lst: List[Any]) -> List[Any]:
1122     seen = set()
1123     new_lst = []
1124     for item in lst:
1125         if item not in seen:

```

```

1126         seen.add(item)
1127         new_lst.append(item)
1128     return new_lst
1129
1130 def sort_dict_by_key(d: Dict[str, Any], reverse: bool = False) -> Dict[str, Any]:
1131     return dict(sorted(d.items(), key=lambda x: x[0], reverse=reverse))
1132
1133 def sort_dict_by_value(d: Dict[str, Any], reverse: bool = False) -> Dict[str, Any]:
1134     return dict(sorted(d.items(), key=lambda x: x[1],
1135
1136 import datetime
1137 from flask import Flask, request, jsonify, g
1138 from flask_sqlalchemy import SQLAlchemy
1139 from flask_marshmallow import Marshmallow
1140 from flask_jwt_extended import JWTManager, create_access_token, jwt_required,
1141 get_jwt_identity
1142 from marshmallow import fields, validate, ValidationError
1143 from functools import wraps
1144 import bcrypt
1145 from sqlalchemy.exc import IntegrityError
1146 import os
1147
1148 app = Flask(__name__)
1149 app.config['SQLALCHEMY_DATABASE_URI'] = os.getenv('DATABASE_URL',
1150 'postgresql://user:password@localhost/defect_detection')
1151 app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
1152 app.config['JWT_SECRET_KEY'] = os.getenv('JWT_SECRET_KEY', 'super-secret-key-1234567890')
1153 app.config['JWT_ACCESS_TOKEN_EXPIRES'] = datetime.timedelta(hours=24)
1154
1155 db = SQLAlchemy(app)
1156 ma = Marshmallow(app)
1157 jwt = JWTManager(app)
1158
1159 class User(db.Model):
1160     id = db.Column(db.Integer, primary_key=True)
1161     username = db.Column(db.String(80), unique=True, nullable=False)
1162     password_hash = db.Column(db.String(120), nullable=False)
1163     role = db.Column(db.String(20), nullable=False, default='user')
1164     created_at = db.Column(db.DateTime, default=datetime.datetime.utcnow)
1165     updated_at = db.Column(db.DateTime, default=datetime.datetime.utcnow,
1166 onupdate=datetime.datetime.utcnow)
1167
1168     def set_password(self, password):
1169         self.password_hash = bcrypt.hashpw(password.encode('utf-8'),
1170 bcrypt.gensalt()).decode('utf-8')
1171
1172     def check_password(self, password):
1173         return bcrypt.checkpw(password.encode('utf-8'), self.password_hash.encode('utf-8'))
1174

```

```

1175 class DefectType(db.Model):
1176     id = db.Column(db.Integer, primary_key=True)
1177     name = db.Column(db.String(50), unique=True, nullable=False)
1178     description = db.Column(db.Text)
1179     created_at = db.Column(db.DateTime, default=datetime.datetime.utcnow)
1180     updated_at = db.Column(db.DateTime, default=datetime.datetime.utcnow,
1181 onupdate=datetime.datetime.utcnow)
1182
1183 class DefectRecord(db.Model):
1184     id = db.Column(db.Integer, primary_key=True)
1185     defect_type_id = db.Column(db.Integer, db.ForeignKey('defect_type.id'), nullable=False)
1186     screen_serial_number = db.Column(db.String(100), nullable=False)
1187     detection_time = db.Column(db.DateTime, default=datetime.datetime.utcnow)
1188     confidence = db.Column(db.Float, nullable=False)
1189     coordinates = db.Column(db.Text)
1190     image_path = db.Column(db.String(255))
1191     detected_by = db.Column(db.Integer, db.ForeignKey('user.id'), nullable=False)
1192     status = db.Column(db.String(20), default='unprocessed')
1193     notes = db.Column(db.Text)
1194     created_at = db.Column(db.DateTime, default=datetime.datetime.utcnow)
1195     updated_at = db.Column(db.DateTime, default=datetime.datetime.utcnow,
1196 onupdate=datetime.datetime.utcnow)
1197
1198     defect_type = db.relationship('DefectType', backref=db.backref('records', lazy=True))
1199     user = db.relationship('User', backref=db.backref('detections', lazy=True))
1200
1201 class DetectionReport(db.Model):
1202     id = db.Column(db.Integer, primary_key=True)
1203     report_number = db.Column(db.String(50), unique=True, nullable=False)
1204     start_time = db.Column(db.DateTime, nullable=False)
1205     end_time = db.Column(db.DateTime, nullable=False)
1206     total_screens = db.Column(db.Integer, nullable=False)
1207     defective_screens = db.Column(db.Integer, nullable=False)
1208     defect_summary = db.Column(db.Text)
1209     generated_by = db.Column(db.Integer, db.ForeignKey('user.id'), nullable=False)
1210     generated_at = db.Column(db.DateTime, default=datetime.datetime.utcnow)
1211     file_path = db.Column(db.String(255))
1212
1213     user = db.relationship('User', backref=db.backref('reports', lazy=True))
1214
1215 class UserSchema(ma.Schema):
1216     id = fields.Integer(dump_only=True)
1217     username = fields.String(required=True, validate=validate.Length(min=3, max=80))
1218     role = fields.String(validate=validate.OneOf(['user', 'admin', 'manager']))
1219     created_at = fields.DateTime(dump_only=True)
1220     updated_at = fields.DateTime(dump_only=True)
1221
1222 class UserRegisterSchema(UserSchema):
1223     password = fields.String(required=True, validate=validate.Length(min=6, max=120))

```

```
1224
1225 class UserLoginSchema(ma.Schema):
1226     username = fields.String(required=True)
1227     password = fields.String(required=True)
1228
1229 class DefectTypeSchema(ma.Schema):
1230     id = fields.Integer(dump_only=True)
1231     name = fields.String(required=True, validate=validate.Length(min=1, max=50))
1232     description = fields.String()
1233     created_at = fields.DateTime(dump_only=True)
1234     updated_at = fields.DateTime(dump_only=True)
1235
1236 class DefectRecordSchema(ma.Schema):
1237     id = fields.Integer(dump_only=True)
1238     defect_type_id = fields.Integer(required=True)
1239     screen_serial_number = fields.String(required=True, validate=validate.Length(min=1,
1240 max=100))
1241     confidence = fields.Float(required=True, validate=validate.Range(min=0.0, max=1.0))
1242     coordinates = fields.String()
1243     image_path = fields.String()
1244     detected_by = fields.Integer(dump_only=True)
1245     status = fields.String(validate=validate.OneOf(['unprocessed', 'processed',
1246 'rejected']))
1247     notes = fields.String()
1248     detection_time = fields.DateTime(dump_only=True)
1249     created_at = fields.DateTime(dump_only=True)
1250     updated_at = fields.DateTime(dump_only=True)
1251
1252 class DetectionReportSchema(ma.Schema):
1253     id = fields.Integer(dump_only=True)
1254     report_number = fields.String(dump_only=True)
1255     start_time = fields.DateTime(required=True)
1256     end_time = fields.DateTime(required=True)
1257     total_screens = fields.Integer(required=True, validate=validate.Range(min=0))
1258     defective_screens = fields.Integer(required=True, validate=validate.Range(min=0))
1259     defect_summary = fields.String()
1260     generated_by = fields.Integer(dump_only=True)
1261     generated_at = fields.DateTime(dump_only=True)
1262     file_path = fields.String()
1263
1264 user_schema = UserSchema()
1265 users_schema = UserSchema(many=True)
1266 user_register_schema = UserRegisterSchema()
1267 user_login_schema = UserLoginSchema()
1268 defect_type_schema = DefectTypeSchema()
1269 defect_types_schema = DefectTypeSchema(many=True)
1270 defect_record_schema = DefectRecordSchema()
1271 defect_records_schema = DefectRecordSchema(many=True)
1272 detection_report_schema = DetectionReportSchema()
```

```
1273 detection_reports_schema = DetectionReportSchema(many=True)
1274
1275 def role_required(required_role):
1276     def decorator(f):
1277         @wraps(f)
1278         @jwt_required()
1279         def decorated_function(*args, **kwargs):
1280             current_user_id = get_jwt_identity()
1281             user = User.query.get(current_user_id)
1282             if not user or user.role != required_role:
1283                 return jsonify({"error": "Insufficient permissions"}), 403
1284             g.user = user
1285             return f(*args, **kwargs)
1286         return decorated_function
1287     return decorator
1288
1289 @app.before_request
1290 def before_request():
1291     if request.method != 'OPTIONS':
1292         if request.path.startswith('/api/auth/'):
1293             return
1294         if request.path == '/api/health':
1295             return
1296
1297 @app.errorhandler(400)
1298 def bad_request(error):
1299     if isinstance(error.description, ValidationError):
1300         return jsonify({"error": "Validation error", "details":
1301 error.description.messages}), 400
1302     return jsonify({"error": "Bad request"}), 400
1303
1304 @app.errorhandler(401)
1305 def unauthorized(error):
1306     return jsonify({"error": "Unauthorized access"}), 401
1307
1308 @app.errorhandler(403)
1309 def forbidden(error):
1310     return jsonify({"error": "Forbidden access"}), 403
1311
1312 @app.errorhandler(404)
1313 def not_found(error):
1314     return jsonify({"error": "Resource not found"}), 404
1315
1316 @app.errorhandler(409)
1317 def conflict(error):
1318     return jsonify({"error": "Resource conflict"}), 409
1319
1320 @app.errorhandler(500)
1321 def internal_error(error):
```

```

1322     db.session.rollback()
1323     return jsonify({"error": "Internal server error"}), 500
1324
1325 @app.route('/api/health', methods=['GET'])
1326 def health_check():
1327     return jsonify({"status": "healthy", "service": "显示屏表面缺陷 AI 智能系统", "version":
1328 "v1.0"}), 200
1329
1330 @app.route('/api/auth/register', methods=['POST'])
1331 def register():
1332     try:
1333         data = user_register_schema.load(request.get_json())
1334     except ValidationError as err:
1335         return jsonify({"error": "Validation error", "details": err.messages}), 400
1336
1337     if User.query.filter_by(username=data['username']).first():
1338         return jsonify({"error": "Username already exists"}), 409
1339
1340     user = User(username=data['username'], role='user')
1341     user.set_password(data['password'])
1342     db.session.add(user)
1343     try:
1344         db.session.commit()
1345     except IntegrityError:
1346         db.session.rollback()
1347         return jsonify({"error": "Database error"}), 500
1348
1349     return jsonify({"message": "User created successfully", "user":
1350 user_schema.dump(user)}), 201
1351
1352 @app.route('/api/auth/login', methods=['POST'])
1353 def login():
1354     try:
1355         data = user_login_schema.load(request.get_json())
1356     except ValidationError as err:
1357         return jsonify({"error": "Validation error", "details": err.messages}), 400
1358
1359     user = User.query.filter_by(username=data['username']).first()
1360     if not user or not user.check_password(data['password']):
1361         return jsonify({"error": "Invalid credentials"}), 401
1362
1363     access_token = create_access_token(identity=user.id)
1364     return jsonify({"access_token": access_token, "user": user_schema.dump(user)}), 200
1365
1366 @app.route('/api/auth/me', methods=['GET'])
1367 @jwt_required()
1368 def get_current_user():
1369     current_user_id = get_jwt_identity()
1370     user = User.query.get(current_user_id)

```

```

1371     if not user:
1372         return jsonify({"error": "User not found"}), 404
1373     return jsonify(user_schema.dump(user)), 200
1374
1375 @app.route('/api/users', methods=['GET'])
1376 @role_required('admin')
1377 def get_users():
1378     users = User.query.all()
1379     return jsonify(users_schema.dump(users)), 200
1380
1381 @app.route('/api/users/<int:user_id>', methods=['GET'])
1382 @role_required('admin')
1383 def get_user(user_id):
1384     user = User.query.get(user_id)
1385     if not user:
1386         return jsonify({"error": "User not found"}), 404
1387     return jsonify(user_schema.dump(user)), 200
1388
1389 @app.route('/api/users/<int:user_id>', methods=['PUT'])
1390 @role_required('admin')
1391 def update_user(user_id):
1392     user = User.query.get(user_id)
1393     if not user:
1394         return jsonify({"error": "User not found"}), 404
1395
1396     try:
1397         data = user_schema.load(request.get_json(), partial=True)
1398     except ValidationError as err:
1399         return jsonify({"error": "Validation error", "details": err.messages}), 400
1400
1401     if 'username' in data:
1402         existing_user = User.query.filter_by(username=data['username']).first()
1403         if existing_user and existing_user.id != user_id:
1404             return jsonify({"error": "Username already exists"}), 409
1405
1406     for key, value in data.items():
1407         setattr(user, key, value)
1408
1409     try:
1410         db.session.commit()
1411     except IntegrityError:
1412         db.session.rollback()
1413         return jsonify({"error": "Database error"}), 500
1414
1415     return jsonify(user_schema.dump(user)), 200
1416
1417 @app.route('/api/users/<int:user_id>', methods=['DELETE'])
1418 @role_required('admin')
1419 def delete_user(user_id):

```

```
1420 user = User.query.get(user_id)
1421 if not user:
1422     return jsonify({"error": "User not found"}), 404
1423
1424 if user.id == g.user.id:
1425     return jsonify({"error": "Cannot delete your own account"}), 403
1426
1427 db.session.delete(user)
1428 try:
1429     db.session.commit()
1430 except IntegrityError:
1431     db.session.rollback()
1432     return jsonify({"error": "Database error"}), 500
1433
1434 return jsonify({"message": "User deleted successfully"}), 200
1435
1436 @app.route('/api/defect-types', methods=['GET'])
1437 @jwt_required()
1438 def get_defect_types():
1439     defect_types = DefectType.query.all()
1440     return jsonify(defect_types_schema.dump(defect_types)), 200
1441
1442 @app.route('/api/defect-types/<int:type_id>', methods=['GET'])
1443 @jwt_required()
1444 def get_defect_type(type_id):
1445     defect_type = DefectType.query.get(type_id)
1446     if not defect_type:
1447         return jsonify({"error": "Defect type not found"}), 404
1448     return jsonify(defect_type_schema.dump(defect_type)), 200
1449
1450 @app.route('/api/defect-types', methods=['POST'])
1451 @role_required('admin')
1452 def create_defect_type():
1453     try:
1454         data = defect_type_schema.load(request.get_json())
1455     except ValidationError as err:
1456         return jsonify({"error": "Validation error", "details": err.messages}), 400
1457
1458     if DefectType.query.filter_by(name=data['name']).first():
1459         return jsonify({"error": "Defect type already exists"}), 409
1460
1461     defect_type = DefectType(**data)
1462     db.session.add(defect_type)
1463     try:
1464         db.session.commit()
1465     except IntegrityError:
1466         db.session.rollback()
1467         return jsonify({"error": "Database error"}), 500
1468
```

```

1469     return jsonify(defect_type_schema.dump(defect_type)), 201
1470
1471 @app.route('/api/defect-types/<int:type_id>', methods=['PUT'])
1472 @role_required('admin')
1473 def update_defect_type(type_id):
1474     defect_type = DefectType.query.get(type_id)
1475     if not defect_type:
1476         return jsonify({"error": "Defect type not found"}), 404
1477
1478     try:
1479         data = defect_type_schema.load(request.get_json(), partial=True)
1480     except ValidationError as err:
1481         return jsonify({"error": "Validation error", "details": err.messages}), 400
1482
1483     if 'name' in data:
1484         existing_type = DefectType.query.filter_by(name=data['name']).first()
1485         if existing_type and existing_type.id != type_id:
1486             return jsonify({"error": "Defect type name already exists"}), 409
1487
1488     for key, value in data.items():
1489         setattr(defect_type, key, value)
1490
1491     try:
1492         db.session.commit()
1493     except IntegrityError:
1494         db.session.rollback()
1495         return jsonify({"error": "Database error"}), 500
1496
1497     return jsonify(defect_type_schema.dump(defect_type)), 200
1498
1499 @app.route('/api/defect-types/<int:type_id>', methods=['DELETE'])
1500 @role_required('admin')
1501 def delete_defect_type(type_id):
1502     defect_type = DefectType.query.get(type_id)
1503     if not defect_type:
1504         return jsonify({"error": "Defect type not found"}), 404
1505
1506     if DefectRecord.query.filter_by(defect_type_id=type_id).first():
1507         return jsonify({"error": "Cannot delete defect type with existing records"}), 409
1508
1509     db.session.delete(defect_type)
1510     try:
1511         db.session.commit()
1512     except IntegrityError:
1513         db.session.rollback()
1514         return jsonify({"error": "Database error"}), 500
1515
1516     return jsonify({"message": "Defect type deleted successfully"}), 200
1517

```

```
1518 @app.route('/api/defect-records', methods=['GET'])
1519 @jwt_required()
1520 def get_defect_records():
1521     page = request.args.get('page', 1, type=int)
1522     per_page = request.args.get('per_page', 20, type=int)
1523     defect_type_id = request.args.get('defect_type_id', type=int)
1524     screen_serial = request.args.get('screen_serial')
1525     start_date = request.args.get('start_date')
1526     end_date = request.args.get('end_date')
1527     status = request.args.get('status')
1528
1529     query = DefectRecord.query
1530
1531     if defect_type_id:
1532         query = query.filter_by(defect_type_id=defect_type_id)
1533     if screen_serial:
1534         query = query.filter(DefectRecord.screen_serial_number.contains(screen_serial))
1535     if start_date:
1536         try:
1537             start_dt = datetime.datetime.fromisoformat(start_date)
1538             query = query.filter(DefectRecord.detection_time >= start_dt)
1539         except ValueError:
1540             return jsonify({"error": "Invalid start_date format"}), 400
1541     if end_date:
1542         try:
1543             end_dt = datetime.datetime.fromisoformat(end_date)
1544             query = query.filter(DefectRecord.detection
1545
1546 import re
1547 import datetime
1548 from typing import Dict, List, Optional, Tuple, Any
1549 from dataclasses import dataclass
1550 from enum import Enum
1551 import psycpg2
1552 import redis
1553 import docker
1554 from docker.errors import DockerException
1555 import requests
1556
1557 class SoftwareCategory(Enum):
1558     APPLICATION = "1 应用软件"
1559     SYSTEM = "2 系统软件"
1560     TOOL = "3 支撑软件"
1561
1562 class PublishStatus(Enum):
1563     PUBLISHED = "是"
1564     UNPUBLISHED = "否"
1565
1566 @dataclass
```

```

1567 class SoftwareCopyrightInfo:
1568     full_name: str
1569     short_name: str
1570     version: str
1571     category: SoftwareCategory
1572     completion_date: datetime.date
1573     is_published: PublishStatus
1574     dev_hardware_env: str
1575     run_hardware_env: str
1576     dev_os: str
1577     dev_tools: str
1578     run_platform: str
1579     support_env: str
1580     programming_languages: str
1581     code_lines: int
1582     dev_purpose: str
1583     target_fields: str
1584     main_functions: str
1585     technical_features: str
1586     company_name: str
1587     unified_social_credit_code: str
1588
1589 class ValidationError(Exception):
1590     pass
1591
1592 class DatabaseError(Exception):
1593     pass
1594
1595 class RedisError(Exception):
1596     pass
1597
1598 class DockerServiceError(Exception):
1599     pass
1600
1601 class ExternalAPICallError(Exception):
1602     pass
1603
1604 class SoftwareCopyrightService:
1605     def __init__(self, db_config: Dict[str, str], redis_config: Dict[str, str], docker_host:
1606 str):
1607         self.db_config = db_config
1608         self.redis_config = redis_config
1609         self.docker_host = docker_host
1610         self.redis_client = redis.Redis(**redis_config)
1611         self.docker_client = docker.DockerClient(base_url=docker_host)
1612
1613     def validate_credit_code(self, credit_code: str) -> bool:
1614         pattern = r'^[0-9A-HJ-NPQRTUWXY]{2}\d{6}[0-9A-HJ-NPQRTUWXY]{10}$'
1615         if not re.match(pattern, credit_code):

```

```

1616         return False
1617     coefficients = [1,3,9,27,19,26,16,17,20,29,25,13,8,24,10,30,28]
1618     chars = '0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ'
1619     total = 0
1620     for i in range(17):
1621         char = credit_code[i]
1622         try:
1623             index = chars.index(char)
1624         except ValueError:
1625             return False
1626         total += index * coefficients[i]
1627     remainder = total % 31
1628     check_char = chars[31 - remainder] if remainder != 0 else '0'
1629     return credit_code[-1] == check_char
1630
1631     def validate_software_info(self, info: Dict[str, Any]) -> None:
1632         errors = []
1633         if not info.get('full_name') or len(info['full_name']) > 200:
1634             errors.append("软件全称不能为空且长度不能超过 200 字符")
1635         if not info.get('short_name') or len(info['short_name']) > 50:
1636             errors.append("软件简称不能为空且长度不能超过 50 字符")
1637         if not info.get('version') or not re.match(r'^V\d+\.\d+$', info['version']):
1638             errors.append("版本号格式不正确，应为 Vx.x 格式")
1639         try:
1640             SoftwareCategory(info.get('category', ''))
1641         except ValueError:
1642             errors.append("软件分类不合法")
1643         try:
1644             completion_date = datetime.datetime.strptime(info.get('completion_date', ''),
1645 '%Y年%m月%d日').date()
1646             if completion_date > datetime.date.today():
1647                 errors.append("开发完成日期不能晚于当前日期")
1648         except ValueError:
1649             errors.append("开发完成日期格式不正确，应为 YYYY 年 MM 月 DD 日")
1650         try:
1651             PublishStatus(info.get('is_published', ''))
1652         except ValueError:
1653             errors.append("是否发表状态不合法")
1654         if not info.get('dev_hardware_env') or len(info['dev_hardware_env']) > 500:
1655             errors.append("开发硬件环境不能为空且长度不能超过 500 字符")
1656         if not info.get('run_hardware_env') or len(info['run_hardware_env']) > 500:
1657             errors.append("运行硬件环境不能为空且长度不能超过 500 字符")
1658         if not info.get('dev_os') or len(info['dev_os']) > 100:
1659             errors.append("开发操作系统不能为空且长度不能超过 100 字符")
1660         if not info.get('dev_tools') or len(info['dev_tools']) > 200:
1661             errors.append("开发环境/工具不能为空且长度不能超过 200 字符")
1662         if not info.get('run_platform') or len(info['run_platform']) > 100:
1663             errors.append("运行平台/操作系统不能为空且长度不能超过 100 字符")
1664         if not info.get('support_env') or len(info['support_env']) > 200:

```

```

1665         errors.append("软件运行支撑环境不能为空且长度不能超过200字符")
1666     if not info.get('programming_languages') or len(info['programming_languages']) >
1667 200:
1668         errors.append("编程语言不能为空且长度不能超过200字符")
1669     code_lines = info.get('code_lines', 0)
1670     if not isinstance(code_lines, int) or code_lines <= 0:
1671         errors.append("源程序量必须为正整数")
1672     if not info.get('dev_purpose') or len(info['dev_purpose']) > 1000:
1673         errors.append("开发目的不能为空且长度不能超过1000字符")
1674     if not info.get('target_fields') or len(info['target_fields']) > 500:
1675         errors.append("面向领域/行业不能为空且长度不能超过500字符")
1676     if not info.get('main_functions') or len(info['main_functions']) > 1000:
1677         errors.append("主要功能不能为空且长度不能超过1000字符")
1678     if not info.get('technical_features') or len(info['technical_features']) > 1000:
1679         errors.append("技术特点不能为空且长度不能超过1000字符")
1680     if not info.get('company_name') or len(info['company_name']) > 100:
1681         errors.append("公司名称不能为空且长度不能超过100字符")
1682     credit_code = info.get('unified_social_credit_code', '')
1683     if not credit_code or not self.validate_credit_code(credit_code):
1684         errors.append("统一社会信用代码格式不正确")
1685     if errors:
1686         raise ValidationError("; ".join(errors))
1687
1688     def transform_to_dataclass(self, raw_data: Dict[str, Any]) -> SoftwareCopyrightInfo:
1689         return SoftwareCopyrightInfo(
1690             full_name=raw_data['full_name'],
1691             short_name=raw_data['short_name'],
1692             version=raw_data['version'],
1693             category=SoftwareCategory(raw_data['category']),
1694             completion_date=datetime.datetime.strptime(raw_data['completion_date'], '%Y年%m
1695 月%d日').date(),
1696             is_published=PublishStatus(raw_data['is_published']),
1697             dev_hardware_env=raw_data['dev_hardware_env'],
1698             run_hardware_env=raw_data['run_hardware_env'],
1699             dev_os=raw_data['dev_os'],
1700             dev_tools=raw_data['dev_tools'],
1701             run_platform=raw_data['run_platform'],
1702             support_env=raw_data['support_env'],
1703             programming_languages=raw_data['programming_languages'],
1704             code_lines=raw_data['code_lines'],
1705             dev_purpose=raw_data['dev_purpose'],
1706             target_fields=raw_data['target_fields'],
1707             main_functions=raw_data['main_functions'],
1708             technical_features=raw_data['technical_features'],
1709             company_name=raw_data['company_name'],
1710             unified_social_credit_code=raw_data['unified_social_credit_code']
1711         )
1712
1713     def save_to_database(self, info: SoftwareCopyrightInfo) -> int:

```

```

1714     conn = None
1715     try:
1716         conn = psycopg2.connect(**self.db_config)
1717         cursor = conn.cursor()
1718         insert_sql = """
1719             INSERT INTO software_copyright (
1720                 full_name, short_name, version, category, completion_date, is_published,
1721                 dev_hardware_env, run_hardware_env, dev_os, dev_tools, run_platform,
1722                 support_env, programming_languages, code_lines, dev_purpose,
1723 target_fields,
1724                 main_functions, technical_features, company_name,
1725 unified_social_credit_code
1726             ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s,
1727 %s, %s, %s, %s)
1728             RETURNING id;
1729         """
1730         cursor.execute(insert_sql, (
1731             info.full_name, info.short_name, info.version, info.category.value,
1732             info.completion_date, info.is_published.value, info.dev_hardware_env,
1733             info.run_hardware_env, info.dev_os, info.dev_tools, info.run_platform,
1734             info.support_env, info.programming_languages, info.code_lines,
1735             info.dev_purpose, info.target_fields, info.main_functions,
1736             info.technical_features, info.company_name, info.unified_social_credit_code
1737         ))
1738         record_id = cursor.fetchone()[0]
1739         conn.commit()
1740         cursor.close()
1741         return record_id
1742     except psycopg2.DatabaseError as e:
1743         if conn:
1744             conn.rollback()
1745             raise DatabaseError(f"数据库操作失败: {str(e)}") from e
1746     finally:
1747         if conn:
1748             conn.close()
1749
1750     def cache_to_redis(self, record_id: int, info: SoftwareCopyrightInfo) -> None:
1751         try:
1752             cache_key = f"software_copyright:{record_id}"
1753             cache_data = {
1754                 "full_name": info.full_name,
1755                 "short_name": info.short_name,
1756                 "version": info.version,
1757                 "category": info.category.value,
1758                 "completion_date": info.completion_date.strftime('%Y-%m-%d'),
1759                 "is_published": info.is_published.value,
1760                 "company_name": info.company_name,
1761                 "unified_social_credit_code": info.unified_social_credit_code
1762             }

```

```

1763         self.redis_client.hset(cache_key, mapping=cache_data)
1764         self.redis_client.expire(cache_key, 86400)
1765     except redis.RedisError as e:
1766         raise RedisError(f"Redis 缓存操作失败: {str(e)}") from e
1767
1768     def check_docker_services(self) -> Dict[str, bool]:
1769         try:
1770             services = ["nginx", "redis", "postgres"]
1771             status = {}
1772             for service in services:
1773                 containers = self.docker_client.containers.list(filters={"name": service},
1774 all=True)
1775                 status[service] = any(container.status == "running" for container in
1776 containers)
1777             return status
1778         except DockerException as e:
1779             raise DockerServiceError(f"Docker 服务检查失败: {str(e)}") from e
1780
1781     def call_credit_code_api(self, credit_code: str) -> Dict[str, Any]:
1782         api_url = "https://api.example.com/credit_code/verify"
1783         try:
1784             response = requests.post(api_url, json={"credit_code": credit_code}, timeout=10)
1785             response.raise_for_status()
1786             return response.json()
1787         except requests.RequestException as e:
1788             raise ExternalAPICallError(f"统一社会信用代码校验API调用失败: {str(e)}") from e
1789
1790     def generate_copyright_report(self, record_id: int) -> str:
1791         try:
1792             cache_key = f"software_copyright:{record_id}"
1793             cached_data = self.redis_client.hgetall(cache_key)
1794             if not cached_data:
1795                 conn = psycopg2.connect(**self.db_config)
1796                 cursor = conn.cursor()
1797                 select_sql = """
1798                     SELECT full_name, short_name, version, company_name, completion_date,
1799                           code_lines, main_functions, target_fields
1800                     FROM software_copyright WHERE id = %s;
1801                 """
1802                 cursor.execute(select_sql, (record_id,))
1803                 row = cursor.fetchone()
1804                 cursor.close()
1805                 conn.close()
1806                 if not row:
1807                     raise DatabaseError(f"未找到ID为{record_id}的软件著作权记录")
1808                 cached_data = {
1809                     "full_name": row[0],
1810                     "short_name": row[1],
1811                     "version": row[2],

```

```

1812         "company_name": row[3],
1813         "completion_date": row[4].strftime('%Y年%m月%d日'),
1814         "code_lines": str(row[5]),
1815         "main_functions": row[6],
1816         "target_fields": row[7]
1817     }
1818     report_content = f"""
1819 软件著作权登记信息采集报告
1820 =====
1821 软件全称: {cached_data['full_name'].decode() if isinstance(cached_data['full_name'], bytes)
1822 else cached_data['full_name']}
1823 软件简称: {cached_data['short_name'].decode() if isinstance(cached_data['short_name'], bytes)
1824 else cached_data['short_name']}
1825 版本号: {cached_data['version'].decode() if isinstance(cached_data['version'], bytes) else
1826 cached_data['version']}
1827 开发单位: {cached_data['company_name'].decode() if isinstance(cached_data['company_name'],
1828 bytes) else cached_data['company_name']}
1829 完成日期: {cached_data['completion_date'].decode() if
1830 isinstance(cached_data['completion_date'], bytes) else cached_data['completion_date']}
1831 源程序量: {cached_data['code_lines'].decode() if isinstance(cached_data['code_lines'], bytes)
1832 else cached_data['code_lines']} 行
1833 面向领域: {cached_data['target_fields'].decode() if isinstance(cached_data['target_fields'],
1834 bytes) else cached_data['target_fields']}
1835 主要功能: {cached_data['main_functions'].decode() if
1836 isinstance(cached_data['main_functions'], bytes) else cached_data['main_functions']}
1837 =====
1838 生成日期: {datetime.datetime.now().strftime('%Y年%m月%d日 %H:%M:%S')}
1839         """.strip()
1840     return report_content
1841 except psycopg2.DatabaseError as e:
1842     raise DatabaseError(f"报告生成失败: {str(e)}") from e
1843 except redis.RedisError as e:
1844     raise RedisError(f"缓存读取失败: {str(e)}") from e
1845
1846 def submit_software_copyright(self, raw_data: Dict[str, Any]) -> Tuple[int, str]:
1847     try:
1848         self.validate_software_info(raw_data)
1849         credit_code = raw_data['unified_social_credit_code']
1850         api_result = self.call_credit_code_api(credit_code)
1851         if not api_result.get('valid', False):
1852             raise ValidationError("统一社会信用代码校验未通过")
1853         docker_status = self.check_docker_services()
1854         if not all(docker_status.values()):
1855             raise DockerServiceError(f"必要服务未运行: {[k for k, v in
1856 docker_status.items() if not v]}")
1857         info = self.transform_to_dataclass(raw_data)
1858         record_id = self.save_to_database(info)
1859         self.cache_to_redis(record_id, info)
1860         report = self.generate_copyright_report(record_id)

```

```

1861         return record_id, report
1862     except ValidationError as e:
1863         raise ValidationError(f"提交失败: {str(e)}") from e
1864     except (DatabaseError, RedisError, DockerServiceError, ExternalAPICallError) as e:
1865         raise type(e)(f"提交失败: {str(e)}") from e
1866
1867     def get_software_info(self, record_id: int) -> Optional[SoftwareCopyrightInfo]:
1868         try:
1869             cache_key = f"software_copyright:{record_id}"
1870             cached_data = self.redis_client.hgetall(cache_key)
1871             if cached_data:
1872                 return SoftwareCopyrightInfo(
1873                     full_name=cached_data['full_name'].decode() if
1874 isinstance(cached_data['full_name'], bytes) else cached_data['full_name'],
1875                     short_name=cached_data['short_name'].decode() if
1876 isinstance(cached_data['short_name'], bytes) else cached_data['short_name'],
1877                     version=cached_data['version'].decode() if
1878 isinstance(cached_data['version'], bytes) else cached_data['version'],
1879                     category=SoftwareCategory(cached_data['category'].decode() if
1880 isinstance(cached_data['category'], bytes) else cached_data['category']),
1881
1882 completion_date=datetime.datetime.strptime(cached_data['completion_date'].decode() if
1883 isinstance(cached_data['completion_date'], bytes) else cached_data['completion_date'], '%Y-
1884 %m-%d').date(),
1885                     is_published=PublishStatus(cached_data['is_published'].decode() if
1886 isinstance(cached_data['is_published'], bytes) else cached_data['is_published']),
1887                     dev_hardware_env="",
1888                     run_hardware_env="",
1889                     dev_os="",
1890                     dev_tools="",
1891                     run_platform="",
1892                     support_env="",
1893                     programming_languages="",
1894                     code_lines=0,
1895                     dev_purpose="",
1896                     target_fields="",
1897                     main_functions="",
1898                     technical_features="",
1899                     company_name=cached_data['company_name'].decode() if
1900 isinstance(cached_data['company_name'], bytes) else cached_data['company_name'],
1901
1902 unified_social_credit_code=cached_data['unified_social_credit_code'].decode

```